

Part I

Edge Impulse: <https://studio.edgeimpulse.com/public/1005239/live>

Idle:

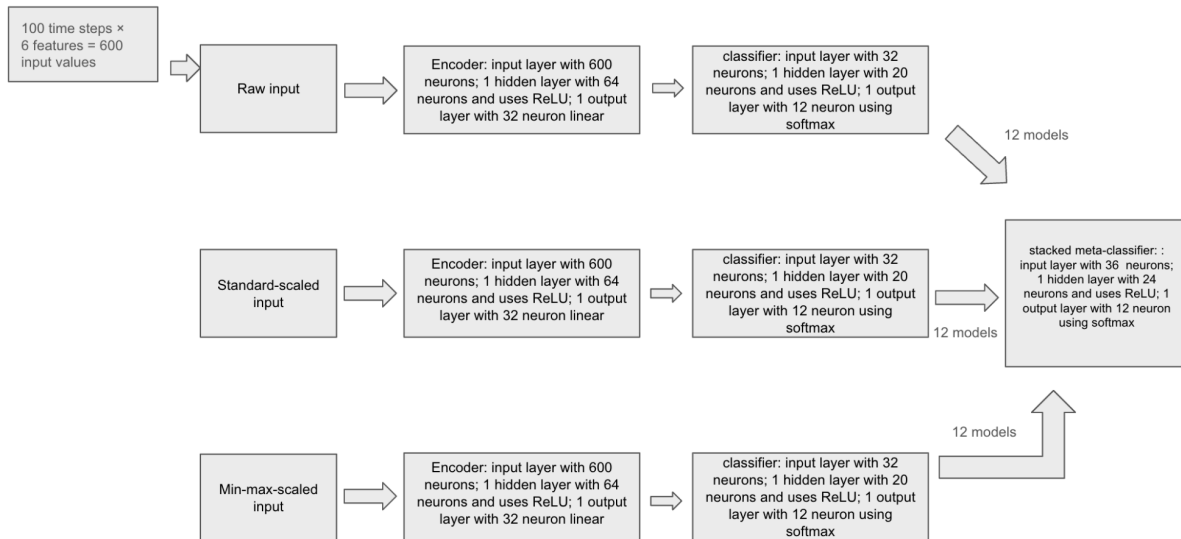
```
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 159 ms, inference 561 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.996094
  lift: 0.000000
```

Lift:

```
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 164 ms, inference 539 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
```

Part II

Question 1:



Question 2:

Pruning is applied with TensorFlow Model Optimization using `prune_low_magnitude()`. The notebook uses a polynomial sparsity schedule, which starts with 10% sparsity and increases to 80% sparsity.

Pruning wrappers are stripped before QAT because the pruning wraps the original Keras layers with extra pruning logic. QAT needs to operate on a cleaner Keras model, so the notebook strips the pruning wrappers while preserves the pruned weight values.

The QAT model is applied after pruning for 5 epochs. The model is simulated with the effects of int8 quantization and learns to tolerate rounding and reduced numerical precision before being exported.

After each training batch and each epoch, the notebook uses an additional mask-enforcement step, which multiplies the QAT weights by the saved pruning masks and makes previously pruned weights to remain zero. After QAT finishes, the callback keeps the model adapts to quantization while still preserving the sparsity created by pruning.

The final model is converted to an int8 TFLite model using `converter.optimizations`. Then it uses a representative dataset, because full integer quantization must estimate the numeric ranges of activations inside the model. These ranges are used to compute the int8 scale and zero-point values. Then it converts the model to use only int8 TensorFlow Lite operations.

Pruning and int8 quantization helps when deploying to a resource-constrained device such as the Arduino Nano 33 BLE Sense: Pruning can reduce storage size and can help if the runtime

or compression format takes advantage of sparsity. Int8 quantization is important because it stores weights and activations as 8-bit integers instead of 32-bit floats, which reduces flash usage and memory. On a board like the Arduino Nano 33 BLE Sense, this makes the model smaller and more efficient for microcontroller inference.

Question 3:

Resting condition output:

```
17:46:27.805 -> Samples collected: 25
17:46:28.334 -> Samples collected: 50
17:46:28.861 -> Samples collected: 75
17:46:29.389 -> Samples collected: 100
17:46:29.389 -> Window collected. Running ensemble inference...
17:46:29.420 -> Raw model scores: 0.0000, 0.0000, 0.9961, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000
17:46:29.420 -> Standard-scaled model scores: 0.0000, 0.0352, 0.9492, 0.0000, 0.0000, 0.0000, 0.0078, 0.0039, 0.0000, 0.0000, 0.0000, 0.0000
17:46:29.420 -> Min-max-scaled model scores: 0.0000, 0.1914, 0.1914, 0.0000, 0.0000, 0.1914, 0.3281, 0.0000, 0.0000, 0.0000, 0.0937, 0.0000
17:46:29.454 -> Meta-classifier scores: 0.1758, 0.0000, 0.6484, 0.0000, 0.0039, 0.1250, 0.0000, 0.0391, 0.0000, 0.0000, 0.0039, 0.0000
17:46:29.454 -> -----
17:46:29.454 -> Final prediction: Lying down
17:46:29.454 -> Class index: 2
17:46:29.454 -> Confidence score: 0.6484
17:46:29.454 -> -----
17:46:29.981 -> Samples collected: 25
17:46:30.509 -> Samples collected: 50
17:46:31.035 -> Samples collected: 75
17:46:31.562 -> Samples collected: 100
17:46:31.562 -> Window collected. Running ensemble inference...
17:46:31.562 -> Raw model scores: 0.0000, 0.0000, 0.9961, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000
17:46:31.562 -> Standard-scaled model scores: 0.0000, 0.0352, 0.9531, 0.0000, 0.0000, 0.0000, 0.0078, 0.0039, 0.0000, 0.0000, 0.0000, 0.0000
17:46:31.593 -> Min-max-scaled model scores: 0.0000, 0.1914, 0.1914, 0.0000, 0.0000, 0.1914, 0.3281, 0.0000, 0.0000, 0.0000, 0.0937, 0.0000
17:46:31.593 -> Meta-classifier scores: 0.1758, 0.0000, 0.6484, 0.0000, 0.0039, 0.1250, 0.0000, 0.0391, 0.0000, 0.0000, 0.0039, 0.0000
17:46:31.593 -> -----
17:46:31.593 -> Final prediction: Lying down
17:46:31.593 -> Class index: 2
17:46:31.593 -> Confidence score: 0.6484
17:46:31.593 -> -----
17:46:32.124 -> Samples collected: 25
17:46:32.650 -> Samples collected: 50
17:46:33.178 -> Samples collected: 75
17:46:33.706 -> Samples collected: 100
17:46:33.706 -> Window collected. Running ensemble inference...
17:46:33.738 -> Raw model scores: 0.0000, 0.0000, 0.9961, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000
17:46:33.738 -> Standard-scaled model scores: 0.0000, 0.0312, 0.9531, 0.0000, 0.0000, 0.0000, 0.0078, 0.0039, 0.0000, 0.0000, 0.0000, 0.0000
17:46:33.738 -> Min-max-scaled model scores: 0.0000, 0.1914, 0.1914, 0.0000, 0.0000, 0.1914, 0.3281, 0.0000, 0.0000, 0.0000, 0.0937, 0.0000
17:46:33.738 -> Meta-classifier scores: 0.1680, 0.0000, 0.6602, 0.0000, 0.0039, 0.1211, 0.0000, 0.0391, 0.0000, 0.0000, 0.0039, 0.0000
17:46:33.771 -> -----
17:46:33.771 -> Final prediction: Lying down
17:46:33.771 -> Class index: 2
17:46:33.771 -> Confidence score: 0.6602
```

The output is mostly acceptable and correctly predicted the lying down condition. However, the confidence score is not very high and the different model gives different scores. One practical change is to collect more data using the device because the original data was collected on human body which may result difference in the predictions.

Question 4:

Yes, sometimes the model predicts lying down even though the device is moving. This may happen because the training data came from body-worn sensors, while the Arduino board is placed on a table. Also, the X, Y, and Z axes of the Arduino IMU may not align with the axes in the original dataset. The model was trained on specific activities with specific movement patterns such as standing, sitting, and lying down, random board movement or a resting board may not match any training activity well.